

3.1 Reducciones polinomiales

 Alumno	 LUIS JAVIER GARNICA ESCAMILLA
 Materia	Analisis de Algoritmos
 Profesor	<u>HECTOR GONZALEZ SANCHEZ</u>
 Status	Listo



UNIVERSIDAD DE
GUADALAJARA

Red Universitaria e Institución Benemérita de Jalisco

Reducciones Polinomiales

Las reducciones polinomiales son una técnica fundamental en la teoría de la complejidad computacional que permite relacionar la dificultad de resolver diferentes problemas.

Definición

Una reducción polinomial es una transformación de un problema A a un problema B, de manera que una solución para B puede ser utilizada para resolver A, y esta transformación se puede realizar en tiempo polinomial.

Tipos de Reducciones

Reducción Many-One ($\leq_m$)

También conocida como reducción Karp, transforma cada instancia del problema A en una única instancia del problema B.

Ejemplo: Reducir el problema del Ciclo Hamiltoniano al problema del Ciclo Hamiltoniano con Vértices Coloreados. La reducción simplemente asigna el mismo color a todos los vértices.

Reducción Turing ($\leq_t$)

Una reducción más general que permite múltiples consultas al problema B para resolver el problema A.

Ejemplo: Reducir el problema de factorización de números al problema de primalidad. Para factorizar un número, podemos hacer múltiples consultas sobre la primalidad de diferentes números.

Reducción Cook ($\leq^c$)

Similar a la reducción Turing, pero con la restricción adicional de que todas las consultas al problema B deben hacerse en paralelo.

Ejemplo: Reducir el problema de determinar si un grafo tiene un conjunto independiente de tamaño k al problema de coloración de grafos.

Propiedades Importantes

- Transitividad: Si $A \leq_p B$ y $B \leq_p C$, entonces $A \leq_p C$
- Preservación de la complejidad: Si $A \leq_p B$ y $B \in P$, entonces $A \in P$
- Herencia de dificultad: Si $A \leq_p B$ y A es NP-completo, entonces B también es NP-completo

Aplicaciones

Las reducciones polinomiales son fundamentales para:

- Demostrar la NP-completitud de problemas
- Establecer relaciones entre problemas computacionales
- Desarrollar algoritmos mediante la transformación de problemas
- Clasificar problemas según su complejidad computacional

```
def subset_sum_to_partition(numbers, target_sum):
    total = sum(numbers)
    new_numbers = numbers + [total - 2 * target_sum]
    return new_numbers

original_numbers = [3, 1, 5, 2]
target = 6
transformed = subset_sum_to_partition(original_numbers, target)
print(f"Conjunto original: {original_numbers}")
print(f"Conjunto transformado: {transformed}")
```